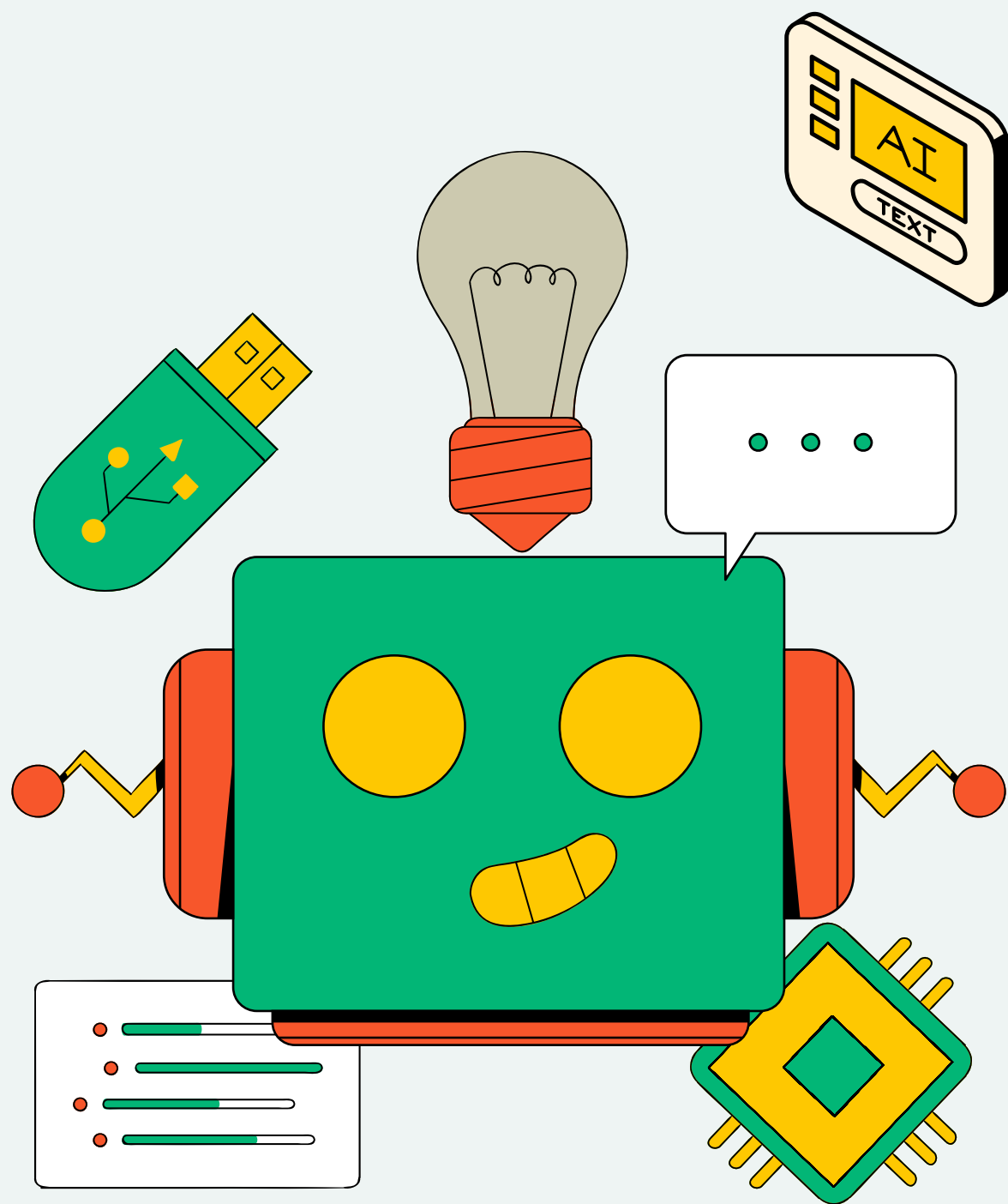


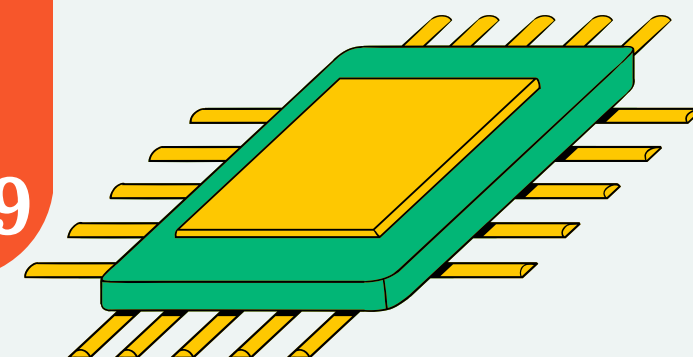
BIG DATA STORAGE AND PROCESSING
GROUP 11



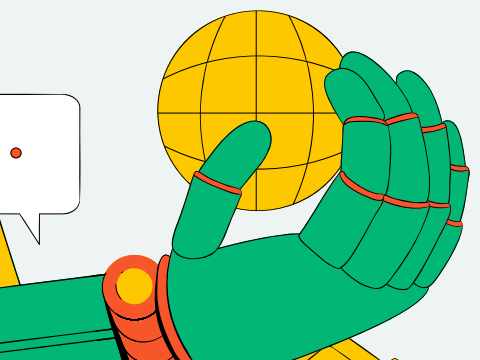
UNITED STATES FLIGHTS DATA PRESENTATION

LECTURER: MR. TRAN VIET TRUNG

VU VIET LONG 20225508
PHAM LONG HAI 20225492
PHAM CONG HIEU 20225493
TRAN VUONG HUNG 20225496
NGUYEN PHAN THANG 20225529

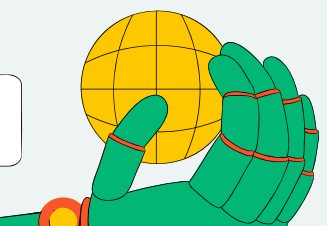


INTRODUCTION



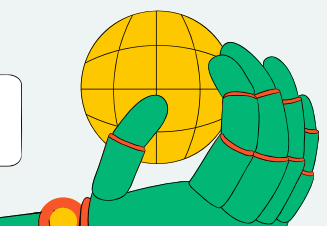
ANALYZING AND MITIGATING FLIGHT DELAYS

- **Problem:** Flight delays and cancellations are significant challenges in the air transportation industry, causing passenger inconvenience, financial losses for airlines, and overall operational inefficiency. These disruptions are caused by various factors including weather, technical issues, and air traffic management.
- **Project Goal:** To use Big Data and real-time processing to proactively address flight disruptions. This project uses a **Lambda architecture** to analyze simulated streaming flight data, combining **historical batch analysis with real-time insights** to enable early detection of potential delays, optimize flight operations, and improve resource allocation.
- **Data Overview:** The project utilizes the **Airline Delay Analysis Dataset from the US Bureau of Transportation Statistics**, containing domestic flight performance data from **2020 to June 2025**. The dataset includes detailed information on flight dates, carriers, origins, destinations, and various delay metrics.

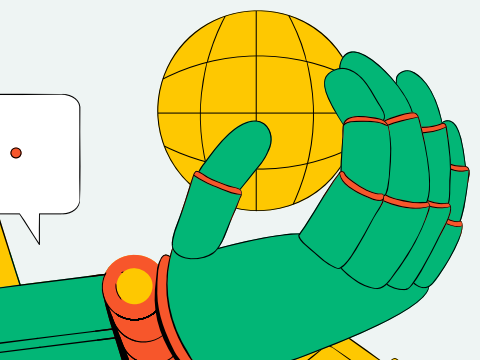


DEPLOYMENT STRATEGY, SCOPE, AND LIMITATIONS

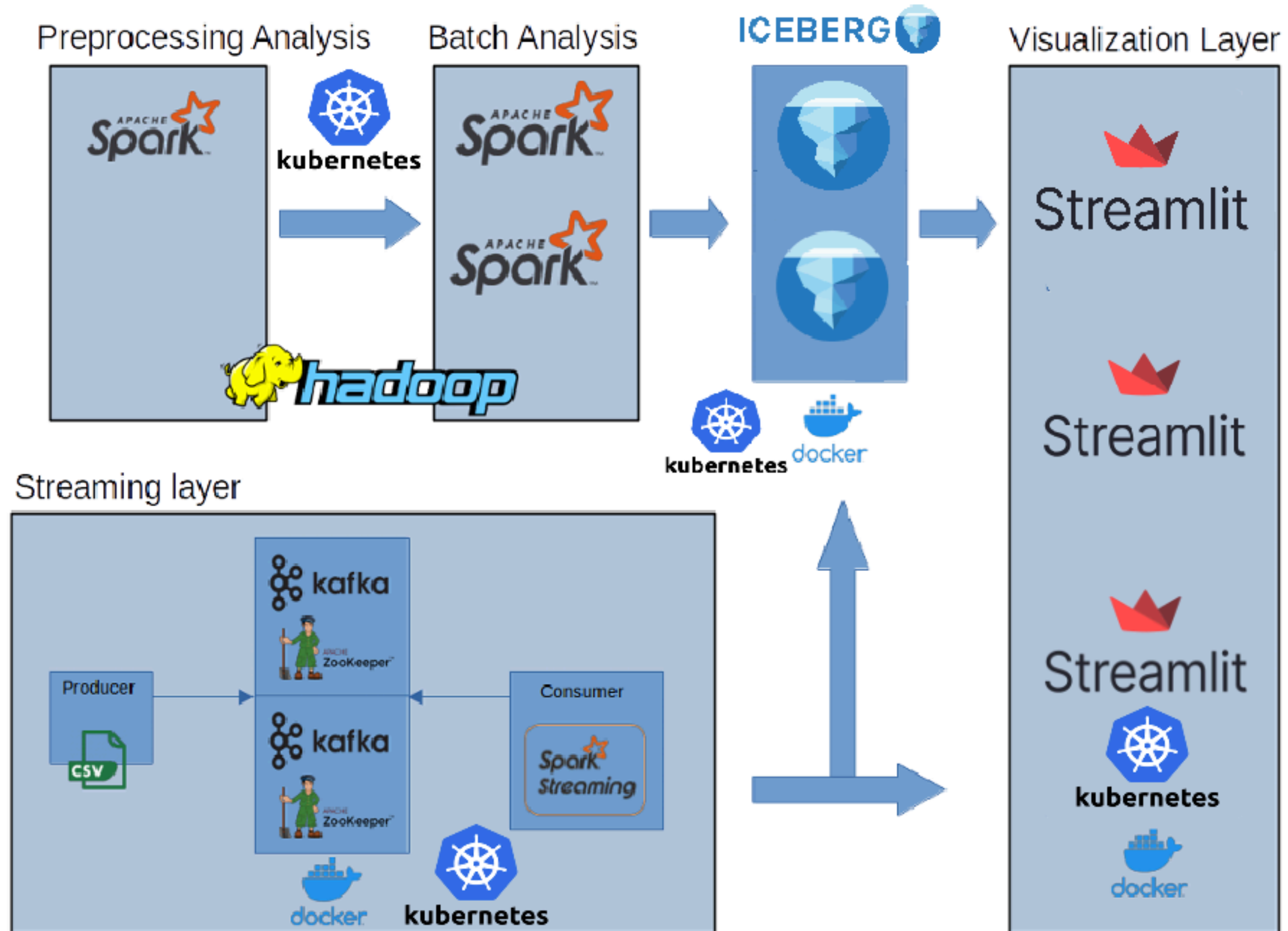
- **Deployment Strategy:**
 - **Phase 1 (Data Acquisition):** Utilized Python scripts to collect data from the US Bureau of Transportation Statistics.
 - **Phase 2 (Local Deployment):** Deployed the entire tech stack using Docker Compose to ensure all components connect and function correctly in a local environment.
 - **Phase 3 (Cloud Migration):** Migrated the system to a **Kubernetes environment**, using Helm to manage the Kafka installation for enhanced computation and analysis on the full dataset.
- **Scope and Limitations:**
 - **Scope:** The project focuses on applying Big Data techniques to analyze US flight data.
 - **Limitation:** The primary challenge is handling the large dataset of nearly 5 million rows. The significant memory required for processing this volume of data restricted the depth and breadth of the analysis that could be performed.



ARCHITECTURE

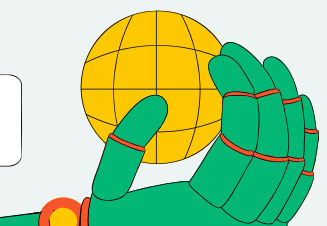


ARTITECHTURE



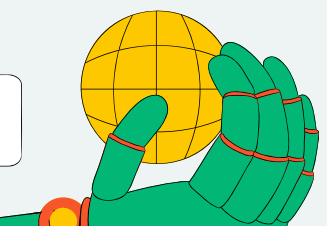
HIGH-LEVEL ARCHITECTURE - THE LAMBDA FRAMEWORK

- **Batch Layer (Historical Analysis)**
 - **Purpose:** To analyze the complete historical dataset for deep, accurate insights and long-term trends.
 - **Process:** Apache Spark processes bulk CSV data, transforming it into structured Apache Iceberg tables stored on HDFS.
 - **Outcome:** Creates reliable, versioned "gold" tables that provide a comprehensive analytical view.
- **Speed Layer (Real-time Processing)**
 - **Purpose:** To provide immediate insights into live flight operations and detect anomalies as they happen.
 - **Process:** A data producer simulates and sends real-time flight events to Apache Kafka. Spark Streaming ingests and processes this data with very low latency.
 - **Outcome:** Delivers real-time statistics and alerts to support rapid decision-making.
- **Serving Layer (Unified View & User Interface)**
 - **Purpose:** To present insights from both the Batch and Speed layers through an interactive dashboard.
 - **Technology:** A Streamlit application queries data directly from Iceberg tables on HDFS.
 - **Features:** Provides both a Real-time Monitoring module (with live KPIs auto-refreshed every 10 seconds) and a Historical Insights module for deep-dive analysis.



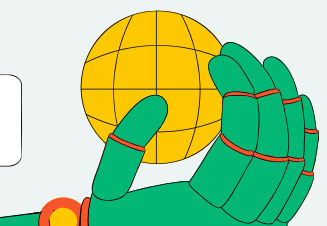
CORE COMPONENTS - DATA PROCESSING & STORAGE

- **Apache Spark: The Unified Analytics Engine**
 - **Role:** Acts as the central processing engine for both the Batch Layer (large-scale aggregations) and the Speed Layer (real-time stream processing).
 - **Deployment:** Runs as a standalone cluster (1 Master, 2 Workers) within a Kubernetes environment for distributed computation.
 - **Integration:** Natively integrates with Apache Iceberg, enabling transactional SQL queries, schema evolution, and time-travel capabilities on data stored in HDFS.
- **HDFS (Hadoop Distributed File System): Scalable & Resilient Storage**
 - **Role:** Serves as the primary distributed storage layer for the entire data lakehouse, holding all historical data and Iceberg tables.
 - **Deployment:** Deployed on Kubernetes with a NameNode (managing metadata) and DataNodes (storing data blocks).
 - **Durability:** Utilizes a Kubernetes PersistentVolumeClaim for the NameNode to ensure that crucial filesystem metadata is preserved across restarts, preventing data loss.

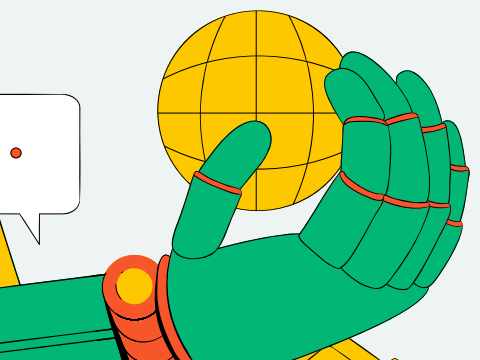


REAL-TIME INGESTION & CONTAINERIZED DEPLOYMENT

- **Apache Kafka: The Real-Time Data Hub**
 - **Role:** Functions as the central ingestion point for the streaming pipeline, managing the flow of live flight data from producers to consumers (Spark Streaming).
 - **Deployment:** Deployed in ZooKeeper mode with a single broker, suitable for a development environment.
 - **Connectivity:** Configured with a dual-listener architecture to allow for both internal (in-cluster) and external communication without compromising network isolation.
 - **Configuration:** Uses PLAINTEXT for simplicity; production environments would require adding SSL/TLS for security.
- **Kubernetes: Container Orchestration**
 - **Role:** Manages the deployment, scaling, and networking of all application components (Spark, HDFS, Kafka, etc.).
 - **Integration:** All components are fully containerized, enabling consistent and reproducible deployments. The architecture uses Kubernetes Services to manage communication between pods.

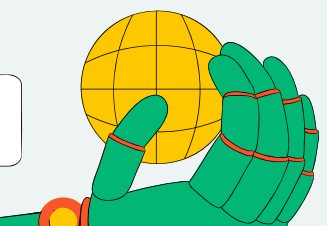


IMPLEMENTATION



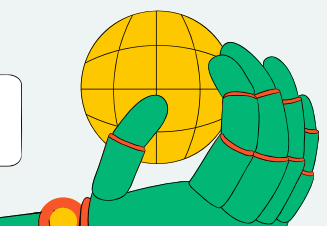
CLUSTER SETUP & DEPLOYMENT ON KUBERNETES

- **Objective:** To create a self-contained, reproducible development environment for the entire data pipeline using Kubernetes.
- **Step 1: Initialize the Local Environment**
 - The process starts by running the `./setup-registry.sh` script.
 - This command creates a local Kubernetes cluster using Kind and sets up a private Docker registry (`localhost:5001`) to store our custom container images.
- **Step 2: Build and Push the Application Image**
 - The Spark application is packaged into a Docker image.
 - This image is then pushed to the local registry, making it available for deployment within the Kubernetes cluster.
- **Step 3: Deploy Components in Order**
 - The entire architecture is deployed using a sequence of Kubernetes manifest files:
 - **01-storage.yaml:** Provisions persistent storage for stateful services like HDFS.
 - **02-infra.yaml:** Deploys the core data infrastructure, including Kafka and ZooKeeper.
 - **03-spark-cluster.yaml:** Deploys the Spark master, worker nodes, and a client pod.
- **Step 4: Verify Deployment**
 - Once all steps are completed, you can verify that all pods for the different components are up and running successfully within the cluster.



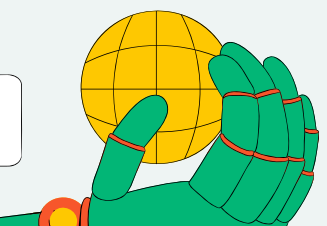
BATCH LAYER IMPLEMENTATION - HISTORICAL ANALYSIS

- **Objective:** To process the entire historical dataset for deep, accurate, and long-term trend analysis.
- **Data Ingestion (`upload_to_hdfs.py`):**
 - A Spark script reads historical flight data from local CSV files.
 - It initializes an Iceberg table named *history_flights* in HDFS, partitioned by year to optimize queries.
 - Data is cleaned, schema-aligned, and appended to the Iceberg table, ensuring transactional, atomic writes.
- **Data Analysis (`batch_analysis.py`):**
 - Another Spark script reads the main *history_flights* table from HDFS.
 - It performs complex aggregations and calculations to answer analytical questions (e.g., "Top 5 worst airlines for delays").
 - The results of each analysis are saved into new, separate **"gold" Iceberg tables**, which are optimized for consumption by the Serving Layer.



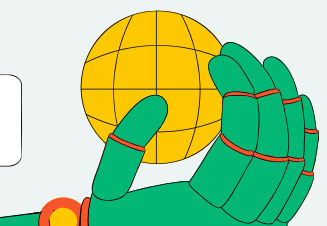
SPEED LAYER: REAL-TIME PROCESSING

- **Objective:** To ingest, process, and store live flight data with minimal latency.
- **Data Producer (producer.py):**
 - A Python script simulates a real-time feed by reading flight data and sending it as JSON messages to an Apache Kafka topic.
 - It sends messages sequentially with a fixed time interval to realistically emulate a continuous data stream.
- **Streaming Ingestion (stream_ingest_hdfs.py):**
 - A Spark Structured Streaming application continuously consumes data from the Kafka topic.
 - It performs real-time data transformations, including parsing, cleaning (handling nulls, converting dates), and schema enforcement.
 - The processed, clean data is written immediately into a partitioned (by day) Iceberg table on HDFS, making it instantly available for real-time querying.



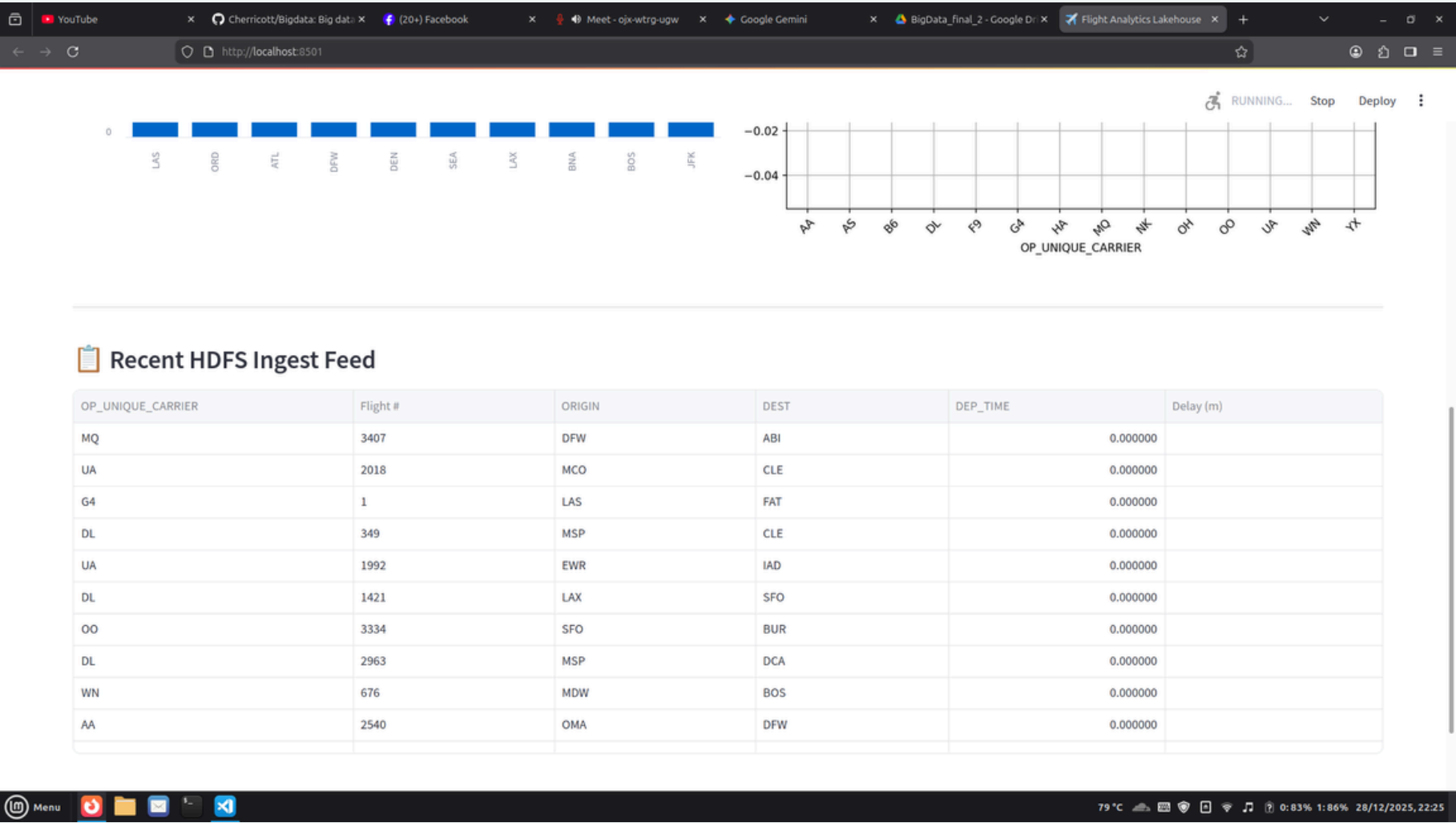
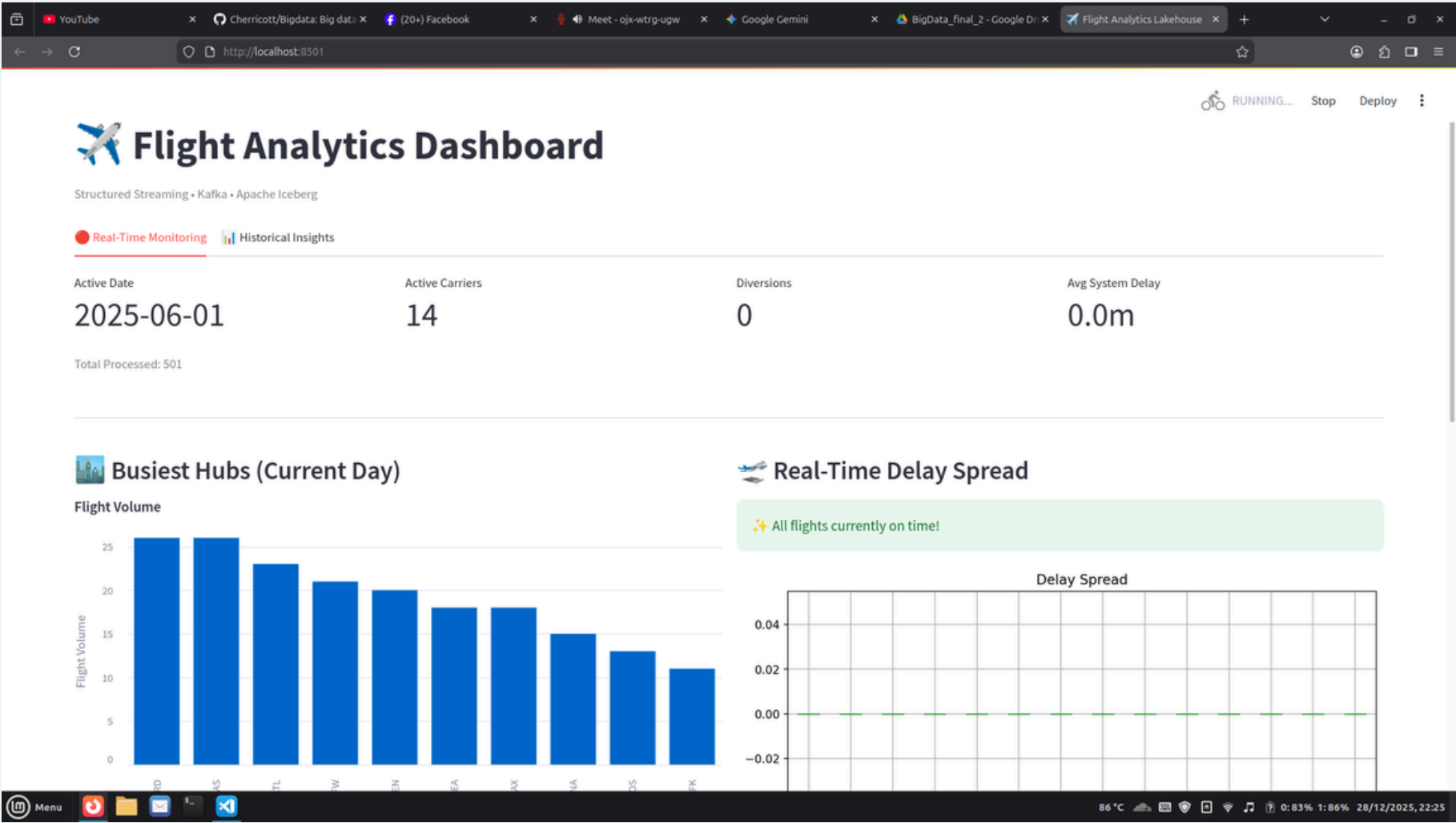
SERVING LAYER IMPLEMENTATION - UNIFIED DASHBOARD

- **Objective:** To provide a unified, interactive interface for both real-time monitoring and historical analysis.
- **Core Framework:**
 - A Streamlit application serves as the user-facing dashboard.
 - It connects directly to HDFS via a persistent, cached SparkSession, ensuring high performance by avoiding repeated initializations.
 - Query results are also cached to reduce latency on repeated requests.



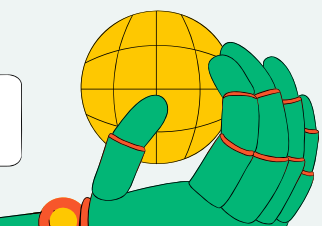
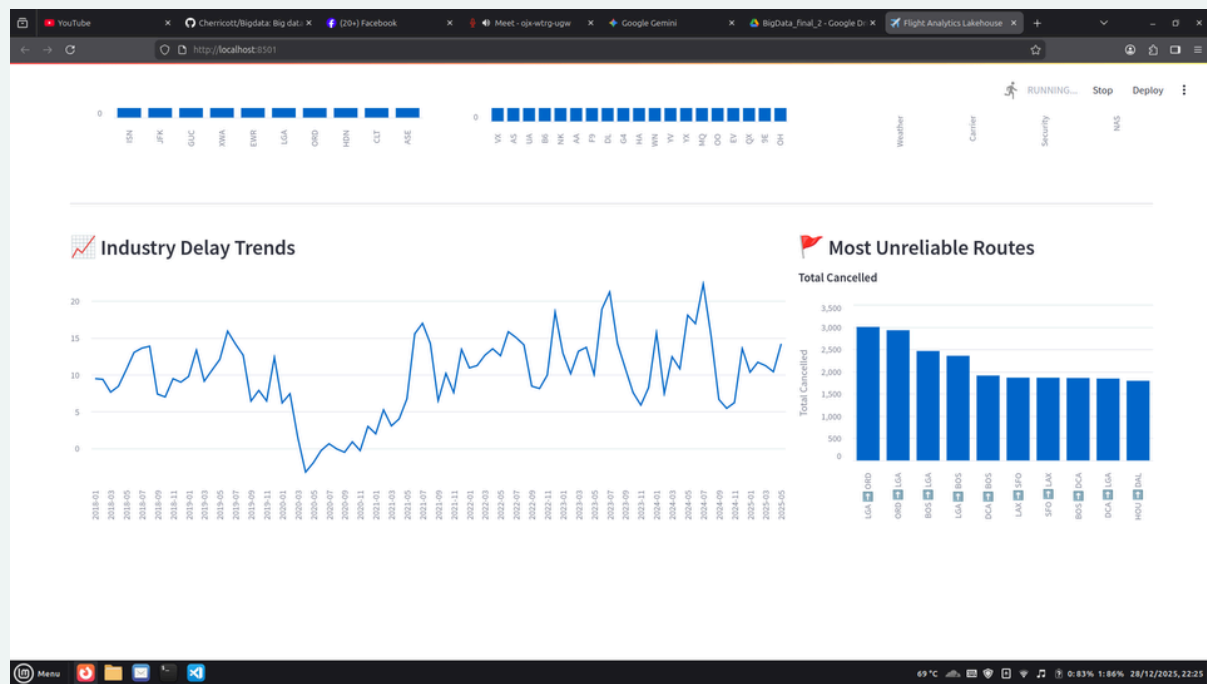
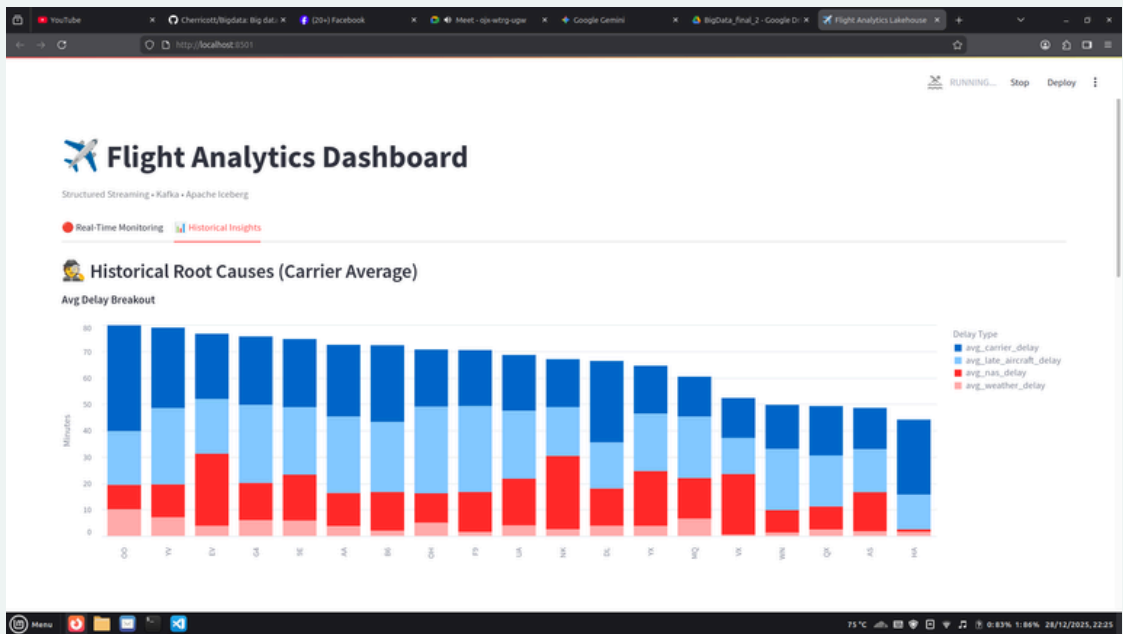
SERVING LAYER IMPLEMENTATION - UNIFIED DASHBOARD

- **Functional Modules: Real-time Monitoring Tab:**
 - Displays live KPIs by querying the latest data from the Speed Layer's Iceberg table.
 - Features an auto-refresh mechanism that re-runs the query every 10 seconds to update visualizations.
 - Includes a "Live Log" table to show raw data as it is ingested.

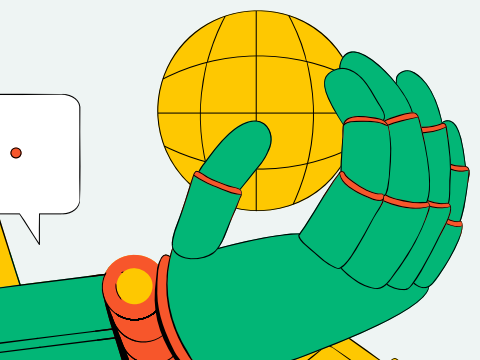


SERVING LAYER IMPLEMENTATION - UNIFIED DASHBOARD

- **Functional Modules: Historical Insights Tab:**
 - Visualizes deep-dive analytics from the Batch Layer's "gold" tables.
 - Uses libraries like Matplotlib to render complex charts (e.g., box plots) for identifying long-term trends and root causes of delays.

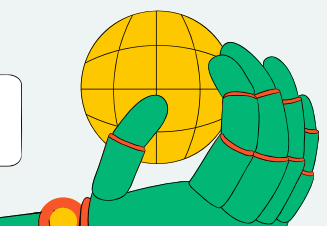


LESSONS LEARNED



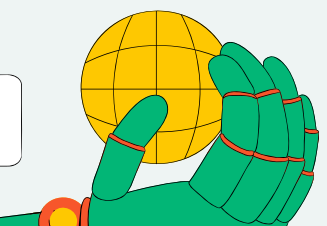
LESSON 1 – MANAGING DUPLICATES AND VERSIONING WITH ICEBERG

- **Problem: Data Inconsistency on Restarts**
 - Restarting the demo caused the Kafka stream to re-process data, leading to double-counting of flights and inaccurate metrics.
 - Storing data as raw files in HDFS made it difficult to undo a faulty ingestion without complex manual cleanup.
- **Solution: Transactional Data Management with Apache Iceberg**
 - Migrated the data storage from raw Parquet files to an Apache Iceberg table format.
 - Implemented a simple pre-shutdown script that runs a DELETE SQL command, removing any data from the most recent streaming session.
 - This ensures each demo starts with a clean slate, preventing overlaps.
- **Results & Key Takeaways**
 - **Result:** Achieved 100% data consistency across restarts and reduced the data footprint by ~70% via Iceberg's built-in compression.
 - **Takeaway:** Use modern table formats like Iceberg or Delta Lake for data pipelines that require idempotency (the ability to re-run without side effects).



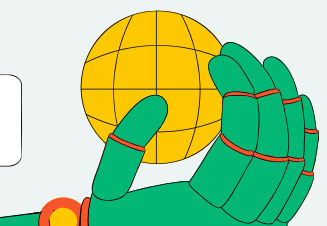
LESSON 2 - OPTIMIZING QUERIES ON LIMITED HARDWARE

- **Problem: Poor Performance on a Large Dataset**
 - Querying the 48.8 million row dataset on a standard laptop was extremely slow, taking over 30 seconds.
 - The Spark cluster was unstable and frequently crashed with Out of Memory (OOM) errors as it tried to scan the entire dataset.
- **Solution: Metadata-Driven Querying and Resource Capping**
 - Leveraged Iceberg's metadata (manifest files), which allows Spark to perform "Data Skipping"—reading only the necessary files instead of scanning the entire table.
 - Set explicit memory limits for Spark (`spark.driver.memory`, `spark.executor.memory`) to prevent it from overwhelming the host machine's resources.
- **Results & Key Takeaways**
 - **Result:** Query execution time dropped dramatically from 30+ seconds to just 4.7 seconds.
 - **Takeaway:** Intelligent data formats that use metadata are often more effective than simply adding more hardware. Capping resources is critical for stability in a containerized environment.



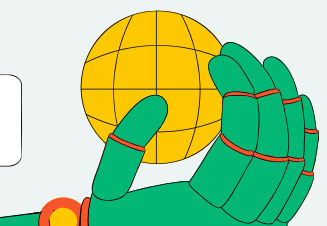
LESSON 3 - ENSURING RELIABILITY IN REAL-TIME STREAMING

- **Problem: Data Loss or Duplication on Stream Failure**
 - When the Spark Streaming client pod crashed, it would lose its place (offset) in the Kafka topic.
 - Upon recovery, the stream would either re-process thousands of records (creating duplicates) or skip a large chunk of data (creating gaps).
- **Solution: Persistent State Management with HDFS Checkpointing**
 - Configured the Spark Structured Streaming job to save its state and current Kafka offset to a persistent checkpoint location in HDFS.
 - If the application crashes, it automatically reads the last saved checkpoint and resumes processing from the exact point it left off.
- **Results & Key Takeaways**
 - **Result:** Achieved true Exactly-Once processing semantics, guaranteeing that every record is processed once and only once, even across failures.
 - **Takeaway:** Checkpointing is non-negotiable for building a reliable, fault-tolerant streaming pipeline.



LESSON 4 – CONNECTING LOCAL TOOLS TO A CONTAINERIZED CLUSTER

- **Problem: "Connection Refused" Errors**
 - Local Python applications (the data producer and the Streamlit dashboard) could not communicate with services like Kafka and HDFS running inside the isolated Kubernetes network.
 - This is because container networks are not directly accessible from the host machine by default.
- **Solution: Network Tunneling with kubectl port-forward**
 - For internal communication (e.g., Spark talking to HDFS), standard Kubernetes service names were used.
 - For external access, kubectl port-forward was used to create a secure tunnel from the local machine directly to the specific service port inside the cluster.
- **Results & Key Takeaways**
 - **Result:** Established stable, three-way communication between local code, the Kafka broker, and the HDFS cluster.
 - **Takeaway:** Tunneling is the most reliable and secure method to bridge local development tools with services running in a containerized environment like Kubernetes.



DASHBOARD DEMO

**THANK YOU FOR
YOUR ATTENTION!**